Advanced CPU Architecture and Hardware Accelerators - Lab 3

Digital System Design with VHDL

(Simple RISC Multi-Cycle CPU design)

Maayan Sarig 206511776

Ido Mayerowicz 207180571

Date: 11.05.2025

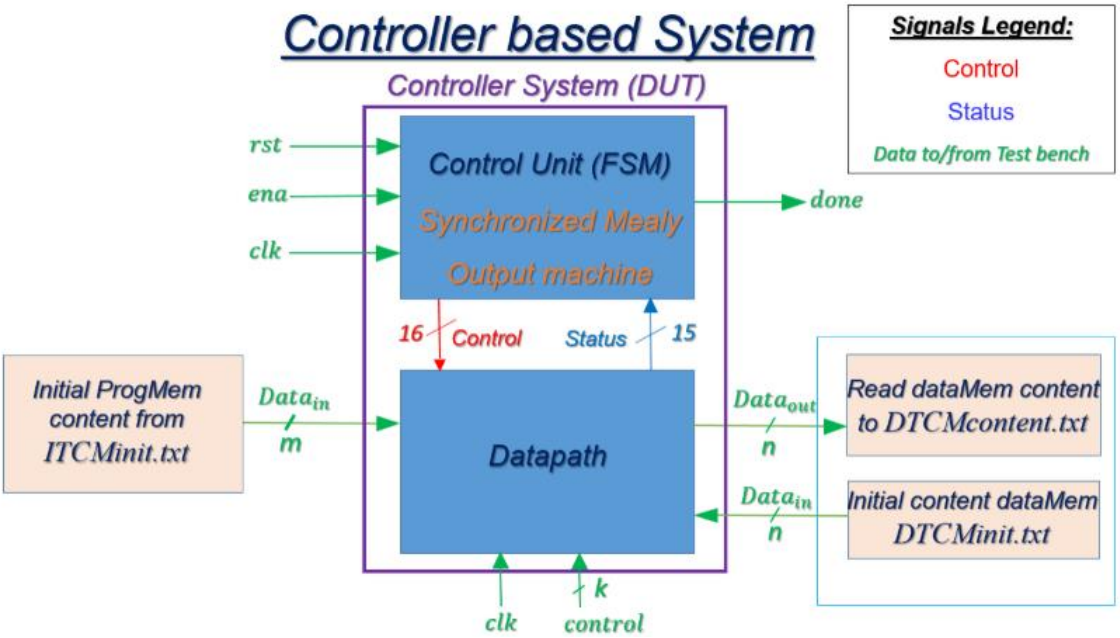
מבוא

מטרות מעבדה מספר 3:

- תכנון מערכת תוך שימוש לוגיקה מקבילית ובדיקה באמצעות שיטות סימולציה מתקדמות.
- תכנון בקר ע"פ מטודולוגיה של הפרדה בין $control$ ובין $datapath$.
- הכנה עבור מעבדה מספר 4.
- ניסיון והבנה של ולידציה וורפיקציה בתכנון ארכיטקטורה.

במעבדה זו נממש מערכת המתבססת על בקר ו- CPU $Multi-cycle$ לצורך הרצה של קוד תוכנית נתונה. את המערכת נממש ע"פ מטודולוגיה של מכונת FSM $Mealy$.

תיאור המערכת

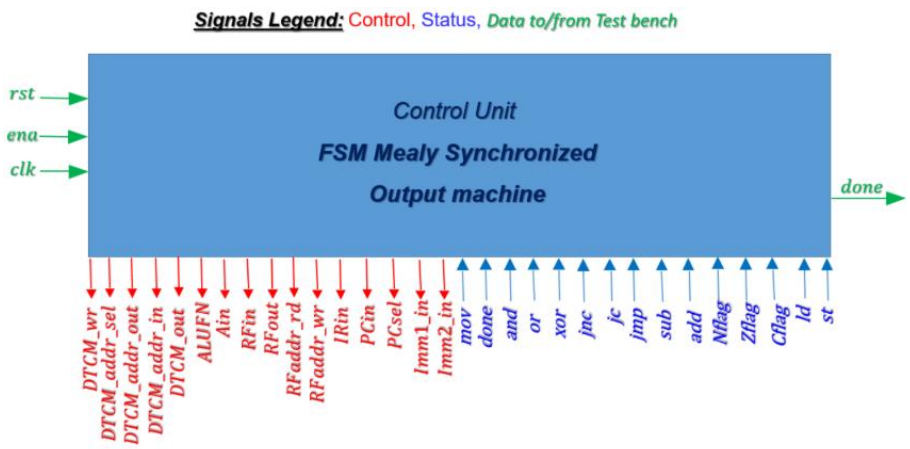


אופן פעולה:

המערכת מקבלת שני קובצים:

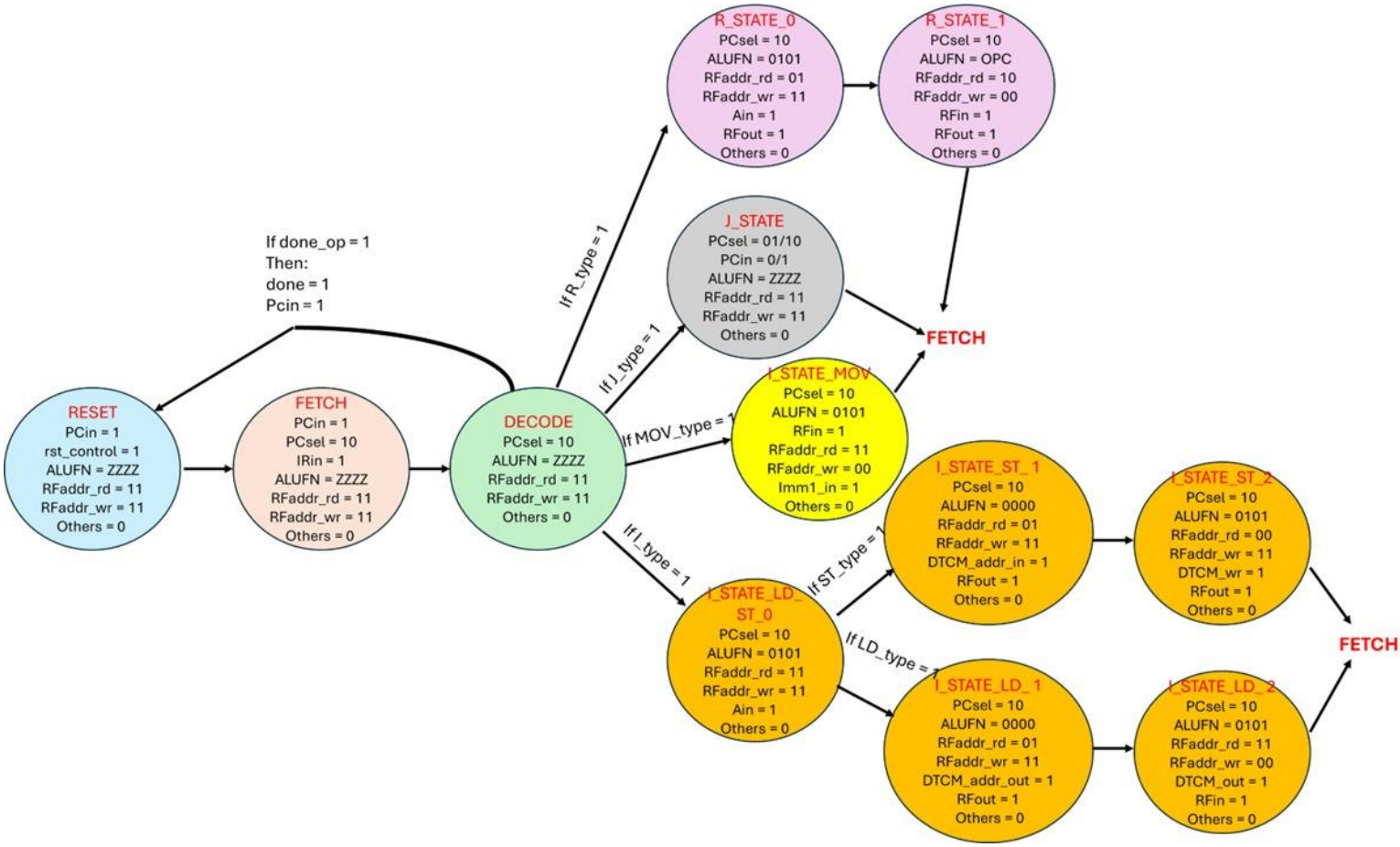
- ITCM$init$ שהינו קובץ הוראות התוכנית.
- DTCM$init$ שהינו קובץ המכיל את המידע בזכרון.

פלט המערכת הינו קובץ המכיל את זכרון המערכת בסיום ריצת התוכנית.



יחידת ה $Control$ אחראית על הדלקה/כיבוי של אתותו הבקרה ע"פ תכנון מכונת המצבים למימוש של התוכנית. היחידה מוציאה קווי בקרה (באדום) לעבר ה $Datapath$, ומקבלת בחזרה קווי חיווי (בכחול) בהתאם לפעולה המתבצעת כעת. בצבע ירוק ישנם אותות כניסה ויציאה אל ה $Test\ bench$.

<u>גרף מצבים למכונת FSM:</u>



- קווי הבקרה RFaddr_rd RFaddr_wr , מצביעים על הרגיסטר הדרוש לפעולה. קבענו את הערך 11 כמצב ביניים בו אין הצבעה לרגיסטר קונקרטי.

גרף מכונת המצבים מתאר את אופן פעולת יחידת ה- control אשר מחליט אילו אותות בקרה יידלקו מתי ובכך מפקח על פעולת ה-$datapath$.

ביצוע תוכנית מתחיל במצב restart ולאחר מכן ביצוע של כל שורת קוד מתחיל בביצוע fetch ואז בשלב ה – decoder אנו מחליטים לאיזה תת-מצב לעבור לפי הפקודה הנדרשת לביצוע. הבדלנו את המצבים השונים:

א) R_type – ביצוע של פקודות חיבור, חיסור והפעולות הלוגיות. מתבצע בעזרת שני מצבים כאשר הראשון מבצע השמה לתוך רגיסטר A והשני מבצע את הפעולה הנדרשת ומבצע השמה לתוך הרגיסטר המבוקש.

ב) J_type – במידה ופעולת ההסתעפות מתקיימת (כלומר יש לבצע את ה-jmp לפי התנאים) אז נבצע השמה לתוך ה-PC של הערך הנוכחי ועוד ה-offset המבוקש שניתן בשדה ה-immediate.

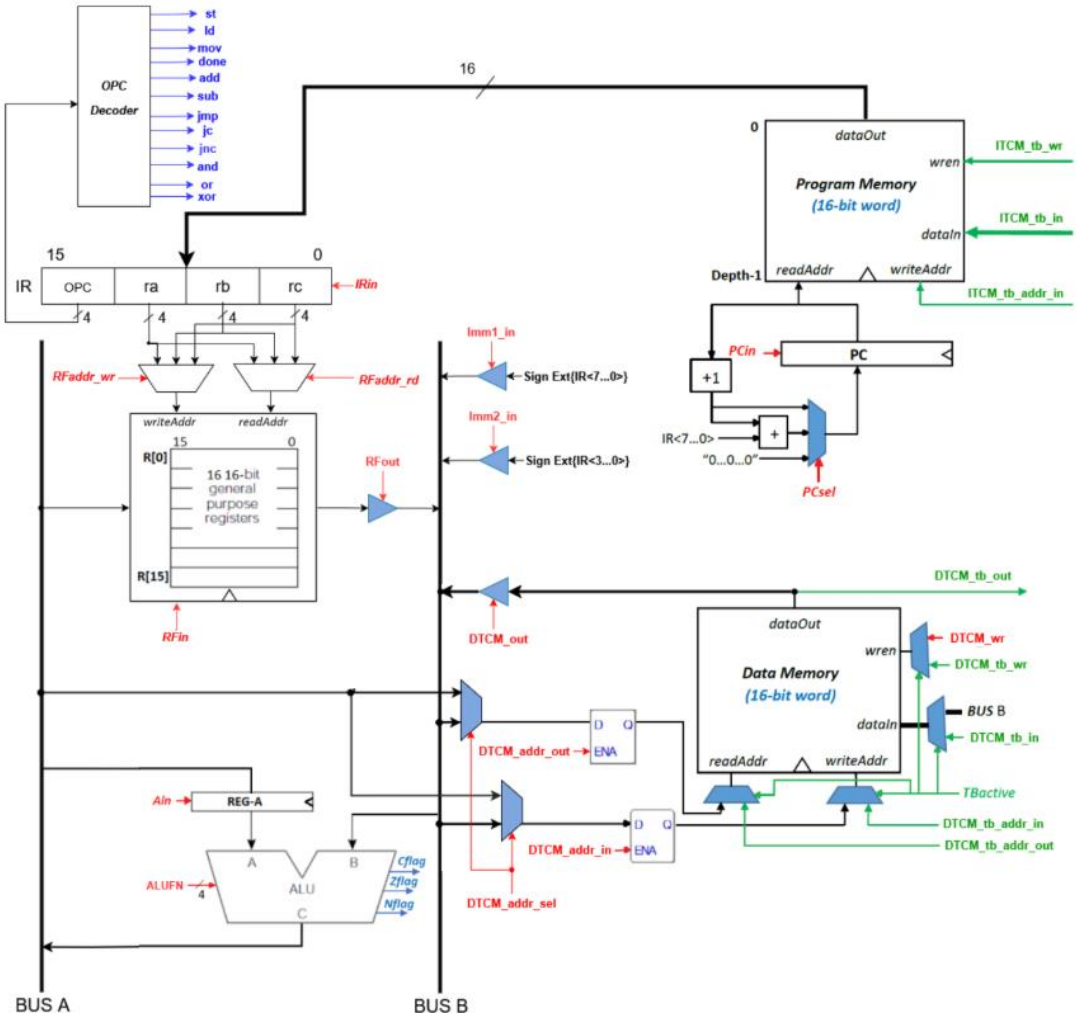
ג) MOV_type – ביצוע של פקודת mov אשר שייכת למשפחת immediate type אך שונה מפעולות ld, st לחלוטין ולכן בעלת מצב משלה. אנו מבצעים כאן קריאה של ערך immediate והכנסתו לתוך הרגיסטר המבוקש.

ד) ld/st_type– כאן אנו רוצים לבצע קריאה או כתיבה לזכרון. לשתי פעולות אלה יש מצב ראשון זהה אשר מבצע חיבור של ערך הרגיסטר למספר offset שנמצא בשדה ה-immediate . משם, מתבצע פיצול לפי ביצוע של כל פעולה-

i. ld – ניקח את ערך הזיכרון שחושב במחזור הקודם אשר ממנו נרצה לבצע קריאה. נכניס אותו לתוך ה-data memory ובמחזור הבא נבצע השמה של הערך מהזכרון לתוך הרגיסטר המבוקש דרך bus – b.

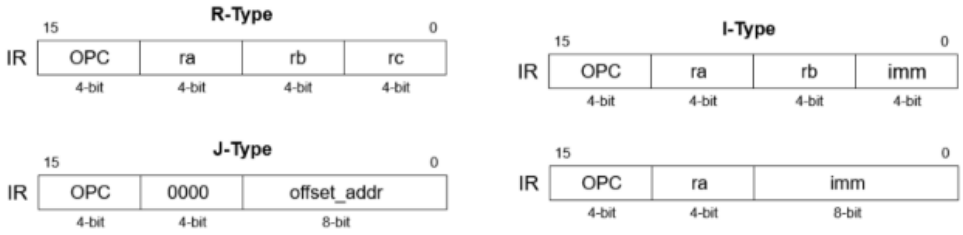
ii. St – ניקח את ערך הזיכרון שחושב במחזור הקודם אשר אליו נרצה לכתוב. נכניס את ערך הכתובת המבוקש ל- data memory ובמחזור הבא נעביר דרך bus – b את הערך אשר צריך לכתוב לזכרון.

דיאגרמת הבלוקים של המערכת



תיאור תרשים הבלוקים

$Instruction\ Register\ (IR)$ – מקבל את ההוראה הנוכחית שהיא מילה באורך 16 bit ומתרגם אותה לפעולות קונקרטיות ע"פ הטבלה הבאה:



יחידת ה OPC Decoder יודעת לפענח את 4 הביטים השמאליים ביותר ולפי הטבלה הנ"ל שולטת באותות הבקרה לזרימת המידע וכן להשמה שלו ע"פ קידוד הפקודה. רגיסטר ה IR מחולק לשדות ומכאן שניתן לפענח את המילה לפקודות קונקרטיות שמנחות את סוג הפקודה, רגיסטרי המקור ורגיסטר היעד.

Instruction Format	Decimal value	OPC	Instruction	Explanation	N	Z	C
R-Type	0	0000	add ra,rb,rc	R[ra]<=R[rb]+R[rc]	*	*	*
			nop	R[0]<=R[0]+R[0] (*emulated instruction*)	*	*	*
	1	0001	sub ra,rb,rc	R[ra]<=R[rb]-R[rc]	*	*	*
	2	0010	and ra,rb,rc	R[ra]<=R[rb] and R[rc]	*	*	-
	3	0011	or ra,rb,rc	R[ra]<=R[rb] or R[rc]	*	*	-
	4	0100	xor ra,rb,rc	R[ra]<=R[rb] xor R[rc]	*	*	-
	5	0101	*unused*				
	6	0110	*unused*				
J-Type	7	0111	jmp offset_addr	PC<=PC+1+offset_addr	-	-	-
	8	1000	jc /jhs offset_addr	If(Cflag==1) PC<=PC+1+offset_addr	-	-	-
	9	1001	jnc /jlo offset_addr	If(Cflag==0) PC<=PC+1+offset_addr	-	-	-
	10	1010	*unused*				
	11	1011	*unused*				
I-Type	12	1100	mov ra,imm	R[ra]<=imm	-	-	-
	13	1101	ld ra,imm(rb)	R[ra] <= M[imm+R[rb]]	-	-	-
	14	1110	st ra,imm(rb)	M[imm+R[rb]] <= R[ra]	-	-	-
Special	15	1111	done	Signals the TB to read the DTCM content	-	-	-

* הוספנו פקודת B = C המתבצעת ב opc = 0101 לצורך שליטה טובה יותר בזרימת המידע.

Register File (RF) - יחידה המכילה 16 רגיסטרים R0 – R15, רגיסטרים אלו משמשים לקריאה וכתיבה של נתונים בזמן ריצת התוכנית. לפי אותות הבקרה יחידת ה RF יודעת מתי ולאיזה רגיסטר להכניס את המידע ובאופן דומה מתי ומאיפה להוציא את המידע.

Arithmetic Logic Unit (ALU) – יחידת החישובים מקבלת את הרגיסטרים עליהם אנו מעוניינים לבצע פעולה מ- Bus A, Bus B ואות בקרה המורה על סוג הפעולה וממשת זאת, לבסוף מוציא את הנתון אל Bus A. כמו כן היחדה הנ"ל מרימה דגלים בהתאם: Cflag, Zflag,Nflag ,

Data Memory (DTCM) – יחידת זכרון סנכרונית המאכסנת את המידע לקריאה וכתיבה. יש לה אותות בקרה המאפשרים לבצע זאת ע"פ קווי הבקרה של יחידת ה control.

Program Counter (PC) – יחידה השומרת את כתובת ההוראה הבאה שצריכה להתבצע. ביחידה זו נממש את קידום התוכנית, כמו כן דרכה נבצע קפיצות לשורות אחרות בתוכנית (jc, jnc, jmp).

Program Memory (PM) – זכרון השומר את פעולות התוכנית, משם מזינים את ה IR. ה PC עוקב אחר המיקום בזכרון זה לביצוע התוכנית הרצויה (כולל קפיצות).

מימשנו את המערכת באמצעות הקבצים הבאים –

- RF – יחידה האחראית על מימוש הרגיסטרים של המעבד.
- $OpcDecoder$ – יחידה אשר מקבלת מספר של 4 ביטים מה-IR ומוציאה ל-control את הסטאטוס.
- $BidirPin$ – יחידה אשר מיישמת tristate בין היחידות השונות לבין ה-bus.
- FA – יחידה המממשת רכיב חיבור.
- top – היחידה הגבוהה ביותר בהיררכיה אשר מקבלת אותות בקרה מבחוץ ומבצעת את התקשורת בין ה-control וה- datapath.
- ALU – יחידה אשר אחראית על ביצוע פעולות חיבור, חיסור ופעולות לוגיות.
- $Aux_package$ – ספרייה שלנו אשר בה מוגדרים ה- components של המערכת.
- $Datapath$ – מגדיר את זרימת הנתונים במערכת, אחראי לנתב את המידע לפי אותות הבקרה שמתקבלים מה control.
- $PCunit$ – מורה על שורת הקוד הבאה כלומר מורה על שורת זכרון בתוך ה- $Program\ Memory$, דרכו מממשים את פקודת ה jump (קפיצה לשורה אחרת בתוכנית).
- $ProgMem$ – יחידת זכרון המכילה את כל ההוראות של התוכנית, ממנה מקבלים פקודות לפענוח ביחידת ה- IR.
- $DataMem$ – יחידת הזכרון, ממנה ניתן לקרוא מידע ואליה ניתן לכתוב את המידע לאחר עיבוד.
- $Control$ – יחידה שאמונה להדליק קווי בקרה לכלל המערכת ע"פ קידוד הפקודה.
- IR – מימוש של יחידת ה- IR שהוצגה מעלה.

תוצאות סימולציה ובדיקות

מוצא המערכת של Test1:

0014
000B
0002
0017
000E
0023
0006
0007
0030
0027
000A
000B
000C
000D
0001

מוצא מערכת תקינה (ע"פ הדרישה):

0014
000B
0002
0017
000E
0023
0006
0007
0030
0027
000A
000B
000C
000D
0001

מוצא המערכת של Test2:

0000	0007
0001	0008
0002	0009
0003	000A
0004	000B
0005	000C
0006	000D
0007	0000
0008	0002
0009	0004
000A	0006
000B	0008
000C	000A
000D	000C
0000	000E
0001	0010
0002	0012
0003	0014
0004	0016
0005	0018
0006	001A

מוצא מערכת תקינה (ע"פ הדרישה):

0000	000A
0001	000B
0002	000C
0003	000D
0004	0000
0005	0002
0006	0004
0007	0006
0008	0008
0009	000A
000A	000C
000B	000E
000C	0010
000D	0012
0000	0014
0001	0016
0002	0018
0003	001A
0004	
0005	
0006	
0007	
0008	
0009	

מוצא המערכת של Test3:

0000	0009
0001	000A
0002	000B
0003	000C
0004	000D
0005	0000
0006	0000
0007	0000
0008	0000
0009	0000
000A	0000
000B	0000
000C	0000
000D	0000
0000	0000
0001	0000
0002	0000
0003	0000
0004	0000
0005	0000
0006	0000
0007	0000
0008	0000

מוצא מערכת תקינה (ע"פ הדרישה):

	0009
	000A
0000	000B
0001	000C
0002	000D
0003	0000
0004	0000
0005	0000
0006	0000
0007	0000
0008	0000
0009	0000
000A	0000
000B	0000
000C	0000
000D	0000
0000	0000
0001	0000
0002	0000
0003	0000
0004	0000
0005	0000
0006	0000
0007	0000
0008	0000

מוצא המערכת של Test4:

0000	000D	000D
0001	000C	000D
0002	000B	0009
0003	000A	0009
0004	0009	000D
0005	0008	000D
0006	0007	0001
0007	0006	0001
0008	0005	000D
0009	0004	000D
000A	0003	0009
000B	0002	0009
000C	0001	000D
000D	0000	000D

מוצא מערכת תקינה (ע"פ הדרישה):

		000D
		0000
		0000
		0000